

AI-POWERED ENTERPRISE MEETING INTELLIGENCE AND ACTION TRACKING PLATFORM

PHASE I REPORT

Submitted by

GIRIJA S P - 230701092

BHUVANESHWARI K - 230701055

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this report titled “AI-POWERED ENTERPRISE MEETING INTELLIGENCE AND ACTION TRACKING PLATFORM” is the bonafide work of **GIRIJA S P (230701092)** and **BHUVANESHWARI K (230701055)**, who carried out the work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. J. Manoranjini M.E., Ph.D.,
Professor and Head,
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai - 602 105.

SIGNATURE

Dr. P. Muneeshwari M.E., Ph.D.,
Associate Professor,
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai - 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Decisions, responsibilities and deadlines in organisations are usually settled through spoken conversation in meetings. Turning that conversation into a usable record normally depends on a participant who listens closely, writes notes and later rewrites them as formal minutes. This is hard to sustain when people are travelling, sitting in a noisy place or simply cannot write and listen at the same time. The AI-Powered Enterprise Meeting Intelligence and Action Tracking Platform tackles this problem. A user submits an audio or video recording of a meeting, and an AI-driven analysis pipeline converts the recording into an organised, reviewable record.

The present implementation is a full-stack web application. Its frontend is built with React and Vite and runs on port 5173; its backend is built with Node.js and Express.js and runs on port 5000. The two communicate through a REST API. The backend receives uploads with Multer, keeps the file temporarily, sends it to the Google Gemini service through the `@google/genai` package, and asks the model for a fixed set of meeting-intelligence fields. The structured JSON reply is saved in a local results file and later fetched and displayed by the frontend.

For every analysed recording the platform produces a summary, key discussion points, decisions, action items (each with a task, an assignee and a due date) and meeting-level deadlines. The report is explicit about scope: real-time transcription, integration with online meeting tools, speaker identification, automatic reminders, multilingual processing and cloud deployment are not part of this phase. No accuracy figure is quoted, because no benchmark dataset or user study has yet been run.

The contribution of the work is the integration of recording upload, backend processing, multimodal AI analysis, structured output, local persistence and a web presentation layer into one enterprise-oriented workflow. It forms a base for later additions such as live transcription, calendar and email links, task reminders, authentication, meeting search, multilingual support, speaker attribution and large-scale evaluation.

Keywords: meeting intelligence, action item extraction, multimodal large language model, Google Gemini, React, Node.js, Express.js, REST API.

ACKNOWLEDGEMENT

We begin by thanking the Almighty for His presence and blessings throughout our lives and during the preparation of this report. Our sincere thanks go to our Chairman Mr. S. MEGANATHAN, B.E., F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing the infrastructure and encouragement that made our education at this institution possible.

We are grateful to our Principal, Dr. S. N. S. MURUGESAN, M.E., Ph.D., for his support and for the facilities that allowed us to complete our work on time. We express our thanks to Dr. J. Manoranjini, M.E., Ph.D., Professor and Head of the Department of Computer Science and Engineering, for her guidance and encouragement during the project. We offer our deepest gratitude to our internal guide, Dr. P. Muneeshwari, M.E., Ph.D., Associate Professor, Department of Computer Science and Engineering, for her valuable guidance at every stage. We also thank our Project Coordinator, Mr. Madhusudhan, B.E., M.E., Assistant Professor, Department of Computer Science and Engineering, for the helpful suggestions he gave during our reviews.

GIRIJA S P 230701092

BHUVANESHWARI K 230701055

TABLE OF CONTENTS

CHAPTER / SECTION NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
1.1	GENERAL	1
1.2	OBJECTIVE	1
1.3	EXISTING SYSTEM	2
1.4	PROPOSED SYSTEM	2
2	LITERATURE SURVEY	3
2.1	LITERATURE REVIEW	3
2.1.1	Meeting data and meeting understanding	3
2.1.2	Speech recognition and speech representation	3
2.1.3	Transformer-based language models	3
2.1.4	Meeting summarisation	4
2.1.5	Actionable items in meetings	4
2.1.6	Multimodal generative models	4
2.2	COMPARISON AND DISCUSSION	5
2.2.1	RESEARCH GAP	6
2.2.2	PROPOSED CONTRIBUTION / NOVELTY	6
2.3	RESEARCH QUESTIONS	6
2.4	CONCLUSION	7
3	SYSTEM DESIGN	8
3.1	SYSTEM ARCHITECTURE	8
3.2	SYSTEM REQUIREMENTS	9
3.2.1	SOFTWARE REQUIREMENTS	9
3.2.2	HARDWARE REQUIREMENTS	9
3.3	API DESIGN	9
3.4	DEPLOYMENT ARCHITECTURE	10
4	PROJECT DESCRIPTION	11
4.1	METHODOLOGY	11

CHAPTER / SECTION NO.	TITLE	PAGE NO.
4.2	MODULES	11
4.2.1	USER / MEETING MANAGEMENT MODULE	11
4.2.2	MEETING RECORDING UPLOAD MODULE	11
4.2.3	AUDIO / VIDEO PROCESSING MODULE	12
4.2.4	AI MEETING ANALYSIS MODULE	12
4.2.5	MEETING SUMMARY MODULE	12
4.2.6	DECISION EXTRACTION MODULE	13
4.2.7	ACTION ITEM EXTRACTION MODULE	13
4.2.8	DEADLINE AND DUE-DATE EXTRACTION MODULE	13
4.2.9	MEETING RESULTS MODULE	14
4.2.10	MEETING HISTORY AND TRACKING MODULE	14
5	IMPLEMENTATION AND RESULT DISCUSSION	16
5.1	FRONTEND IMPLEMENTATION	16
5.2	BACKEND IMPLEMENTATION	16
5.3	APPLICATION ROUTES AND API OPERATIONS	16
5.4	GEMINI AI INTEGRATION	17
5.5	MEETING RECORDING UPLOAD	17
5.6	AI PROCESSING WORKFLOW	17
5.7	END-TO-END SYSTEM DEMONSTRATION	18
5.8	FUNCTIONAL TESTING	20
5.9	SYSTEM OUTPUTS	21
5.10	IMPLEMENTATION DISCUSSION AND OBSERVED ISSUES	21
5.11	LIMITATIONS	22
5.12	SUMMARY OF CONTRIBUTIONS	23
6	CONCLUSION AND FUTURE WORK	24
6.1	CONCLUSION	24
6.2	FUTURE WORK	24
	REFERENCES	26

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Summary of reviewed literature	5
3.1	Software requirements	9
3.2	Hardware requirements	9
3.3	Input and output description of API operations	10
4.1	Module implementation summary	15
5.1	Application route summary (frontend, localhost:5173)	17
5.2	Functional test cases	20
5.3	Output fields of the implemented AI result	21
5.4	Observed issues and corrective considerations	22

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
5.1	Meeting Upload Screen	18
5.2	AI Processing Screen	19
5.3	Meeting Summary	19
5.4	Action Items	19

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AMI	Augmented Multi-party Interaction
API	Application Programming Interface
ASR	Automatic Speech Recognition
CSS	Cascading Style Sheets
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
NLP	Natural Language Processing
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single Page Application
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

CHAPTER 1 INTRODUCTION

1.1 GENERAL

Meetings are one of the main ways in which organisations plan work, hand out responsibilities, review progress, settle problems and take decisions. A single meeting often carries several kinds of information at once: a problem is raised, a solution is agreed, a task is given to a colleague, a target date is mentioned and some follow-up work appears, all within a few minutes of conversation. If that information survives only in the raw recording or in a participant's partial notes, finding it again and acting on it later becomes difficult.

The difficulty grows when employees cannot take notes comfortably. Someone may join while travelling, sit in a noisy environment, or need to focus on the discussion rather than on writing minutes. An unstable network can also interrupt participation in an online meeting. These situations create a clear need to keep the content of a meeting and convert it into an organised form that can be reviewed afterwards.

The AI-Powered Enterprise Meeting Intelligence and Action Tracking Platform was designed for this need. It accepts an audio or video recording of a meeting and passes it through a backend pipeline. The backend keeps the uploaded media temporarily, sends it to the configured Gemini service and requests structured meeting intelligence. The reply is organised into a summary, key discussion points, decisions, action items, assignees, due dates and deadlines, and the frontend presents it to the user.

The current phase is a recording-based analysis system and not a live meeting assistant. It does not transcribe in real time, join online meeting platforms, identify speakers, send reminders or synchronise with calendars and email. These capabilities are treated as future extensions.

1.2 OBJECTIVE

The aim of the project is to build an enterprise-oriented web platform that turns a recorded meeting into structured information that is easier to review and follow up. The specific objectives are:

- 1 To offer a web interface through which a user can submit a meeting audio or video recording.
- 2 To carry the recording from the React frontend to the Node.js and Express.js backend through a REST-based interface.
- 3 To analyse the recording with the Google Gemini API using the `@google/genai` package.

- 4 To generate a structured result containing a summary, key discussion points, decisions, action items, assignees, due dates and deadlines.
- 5 To save the result in the local result store so that the frontend can retrieve and display it.
- 6 To keep the architecture modular so that real-time transcription, enterprise integrations, reminders, authentication, search and cloud deployment can be added later.

1.3 EXISTING SYSTEM

In most workplaces, meeting records still depend on manual note taking or on plain storage of the recording. With manual notes, one or more participants write down the important points during the meeting and later edit them into minutes for circulation. The quality of those minutes depends on the attentiveness, experience and availability of the note taker.

Storing the full audio or video file keeps the source material safe, but it does not sort the content into decisions or tasks. A person who needs one particular action item must scrub through the recording to find it. Separate transcription or summarisation tools reduce the effort of producing text, yet the user must still work out which passages are decisions, which are tasks, who owns them and when they are due.

1.4 PROPOSED SYSTEM

The proposed system places an AI-based processing stage between the recording and the final meeting record. The user uploads a recording in the React frontend, which forwards it to the Express backend. Multer handles the multipart upload and the backend stores the file temporarily. The backend then sends the file to Gemini through `@google/genai` together with an instruction that describes the output structure that is required.

Gemini analyses the media and returns the result. The backend expects the fields `summary`, `keyPoints`, `decisions`, `actionItems` and `deadlines`, where every action item holds a task, an assignee and a `dueDate`. The backend saves the result in the current data store and the frontend retrieves it and displays each part separately.

The architecture is deliberately split into presentation, application, AI-service and storage responsibilities. Because of this split, the local file store could later be replaced by a database without altering the frontend-to-AI flow, and dedicated speech preprocessing could be inserted before the Gemini stage.

CHAPTER 2 LITERATURE SURVEY

This chapter reviews the research and technical material that surrounds the proposed platform. It covers meeting corpora, speech recognition, Transformer-based language models, meeting summarisation, detection of actionable items, large-language-model approaches to meeting summaries and multimodal model documentation. The review is meant to place the project in its technical context; the project does not reproduce any of the cited studies.

2.1 LITERATURE REVIEW

2.1.1 Meeting data and meeting understanding

Carletta et al. [1] introduced the AMI Meeting Corpus, a collection of recorded multi-party meetings with synchronised audio and video and several layers of human annotation. The corpus shows how rich a meeting is as a data source, since it combines speech, interaction between several people and task-oriented content, and it became a reference resource for computational meeting research.

2.1.2 Speech recognition and speech representation

Radford et al. [2] trained the Whisper model on a very large amount of weakly supervised audio and showed that one model can transcribe speech robustly across many conditions without task-specific tuning. Baevski et al. [3] proposed wav2vec 2.0, which learns speech representations from unlabelled audio and then needs only a small amount of labelled data to reach good recognition quality. Together these works show that spoken content can be converted into usable text or representations reliably, which is the first requirement for analysing meeting recordings.

2.1.3 Transformer-based language models

Vaswani et al. [4] replaced recurrent processing with an attention-only architecture, the Transformer, which can relate distant parts of a long sequence and trains efficiently in parallel. Devlin et al. [5] built on this idea with BERT, a model pre-trained to use context from both directions, which improved performance across many language-understanding tasks. These two papers form the basis of the large language models used in current meeting-analysis work.

2.1.4 Meeting summarisation

Zhong et al. [6] presented QMSum, a benchmark for query-based summarisation of meetings drawn from several domains. Their work highlights that meeting transcripts are long, loosely structured and full of information that does not matter for a given purpose, so selecting and condensing the right parts is difficult. Laskar et al. [8] discussed how large language models can be used to build practical meeting-summarisation systems in an industrial setting, including how to cope with long transcripts and with deployment constraints. Kirstein et al. [9] examined LLM-based abstractive summarisation that draws on several sources and can be tailored to the reader's needs, showing that summaries can be shaped to what a particular person must know.

2.1.5 Actionable items in meetings

Chen and Hakkani-Tür [7] studied the detection of actionable items in meetings, that is, the parts of the discussion in which a task is proposed or accepted, together with details such as who is responsible and when the work should be done. Their work establishes action detection as a distinct problem within meeting understanding and relates directly to the action-item, assignee and due-date fields of this project.

2.1.6 Multimodal generative models

The Gemini report [10] describes a family of models trained jointly on text, images, audio and video. The public Gemini API documentation on audio understanding [11] and video understanding [12] explains that such models can be asked to transcribe, summarise or answer questions about recorded audio and video. This capability allows a recording to be submitted directly for analysis without building a separate local recognition pipeline, and it is the technical basis for the AI stage of the proposed platform.

2.2 COMPARISON AND DISCUSSION

Table 2.1. Summary of reviewed literature

Ref.	Focus of the work	Relevance to the proposed platform
[1]	AMI Meeting Corpus: recorded and annotated multi-party meetings	Shows the multimodal and multi-speaker nature of meeting data
[2], [3]	Speech recognition and speech representation learning	Show that spoken content can be processed reliably
[4], [5]	Transformer and BERT language models	Foundation of modern language understanding
[6]	QMSum benchmark for meeting summarisation	Highlights the difficulty of summarising long meetings
[7]	AIMU: detecting actionable items	Closest to action-item, assignee and due-date extraction
[8], [9]	LLM-based meeting summarisation	Support using large language models for practical meeting analysis
[10]–[12]	Gemini models and API documentation	Support direct analysis of audio and video by a multimodal model

Reading these works together, three themes shaped the design of the platform. The first is that recorded meetings contain several types of information at once: discussion, decisions and task-related commitments. The AMI corpus [1] and the speech-recognition studies [2], [3] show that such content can be captured and processed. The second is that language-model research [4]–[6], [8], [9] has shown that long conversational content can be condensed into useful summaries, which supports the use of a multimodal generative model to produce the summary and the key points. The third is that extracting tasks from conversation is an important part of meeting understanding [7], which matches the platform’s extraction of tasks, owners and due dates.

The reviewed works differ from this project mainly in scope. The AMI corpus is a research dataset rather than an application. Whisper and wav2vec 2.0 address recognition and representation, not the full analysis of a meeting. The Transformer and BERT papers supply foundations but no meeting-management tool. QMSum is a benchmark for summarisation, and AIMU concentrates on finding actionable items. The LLM-based summarisation studies [8], [9] examine how large models can be used for meetings but do not describe the frontend-to-backend workflow built here. The platform in this report therefore brings upload, backend processing, multimodal analysis, structured output and presentation into a single workflow. Its contribution is the integration of existing AI capabilities into an application that converts a recorded

meeting into structured, actionable information, not a new recognition or language model.

2.2.1 RESEARCH GAP

The literature shows strong progress on individual components of meeting understanding: recognition, summarisation and action detection. Each study, however, is generally centred on one model, dataset, benchmark or task. The gap addressed here is one of application-level integration. The project combines recording upload, backend processing, multimodal analysis, structured JSON output, local storage and a web interface in one workflow.

The gap is therefore not a lack of research on meeting summarisation. It is the practical combination of several capabilities in a compact enterprise-oriented application that yields explicit fields for summary, key points, decisions, actions, assignees, due dates and deadlines, without claiming a new recognition or language-model algorithm.

2.2.2 PROPOSED CONTRIBUTION / NOVELTY

The contribution is mainly at system level. First, the platform offers an end-to-end path from a recording to meeting intelligence. Second, it requests structured output instead of free-form prose, so that each category of information can be displayed on its own and reused by later features. Third, it combines a React frontend, an Express backend, Multer-based file handling, Gemini multimodal analysis and local result storage into one meeting-management application. The novelty is thus one of integration and application, not of a new machine-learning method.

2.3 RESEARCH QUESTIONS

- 1 Can a web-based ingestion pipeline carry meeting audio or video to an AI service and return a structured meeting-intelligence result?
- 2 Can a multimodal large language model organise meeting content into a summary, key points, decisions, action items, assignees, due dates and deadlines?
- 3 Can a modular React and Express architecture provide a practical interface for retrieving and presenting structured meeting results?
- 4 What limitations and extension needs appear when a recording-based workflow is used as the foundation for enterprise meeting management?

2.4 CONCLUSION

The literature supplies the technical grounding for the platform. Meeting corpora illustrate the complexity of multi-party data; speech research shows how audio can be turned into usable information; Transformer and language-model research provides contextual understanding; and meeting-specific studies underline the value of summaries and action items. The project builds on these areas by bringing multimodal AI analysis into a web-based meeting workflow. Its present contribution is practical and system-oriented, while questions such as multilingual evaluation, speaker attribution, benchmark accuracy and large-scale enterprise validation are left for future work.

CHAPTER 3 SYSTEM DESIGN

This chapter describes the architecture and engineering requirements of the platform. The design is client-server: the React frontend provides the interface, the Node.js and Express.js backend exposes the REST API and manages file handling, the Gemini service performs the AI analysis, and a local JSON-based store keeps the generated results.

3.1 SYSTEM ARCHITECTURE

The system is organised in layers. In the presentation layer, a React application built with Vite offers the meeting-management interface on localhost:5173. In the application layer, Node.js with Express.js provides the REST API on localhost:5000. The backend uses Multer to receive the uploaded recording and stores it temporarily in an uploads directory. In the AI-service layer, the Google Gemini API is accessed through the `@google/genai` package. In the storage layer, generated results are written to the file `data/results.json`.

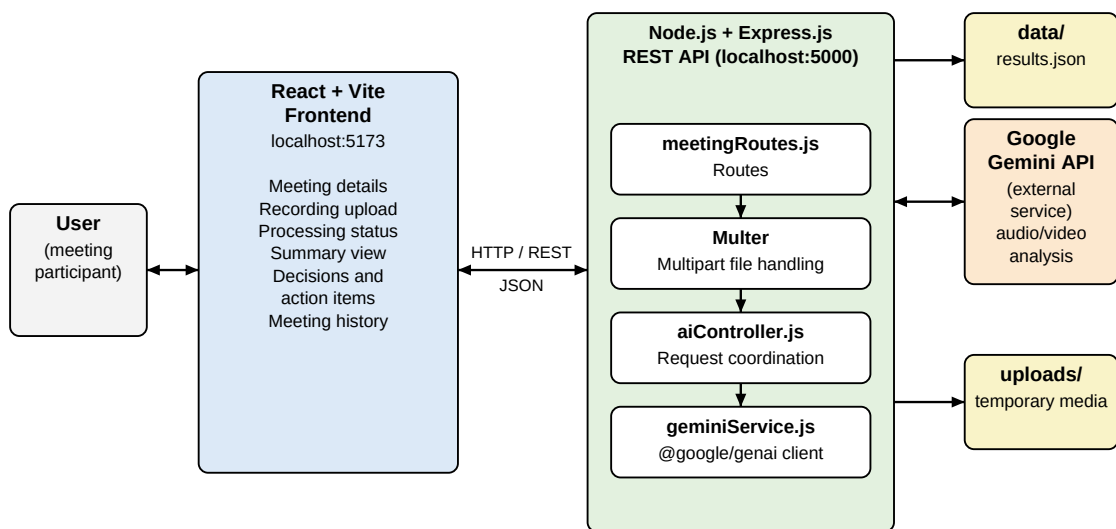


Figure 3.1. System architecture of the platform

Figure 3.1 shows how a request moves through these layers. The user works in the React frontend, which exchanges HTTP and JSON messages with the Express backend. Inside the backend, `meetingRoutes.js` receives the request, `Multer` stores the uploaded file in `uploads/`, `aiController.js` coordinates the operation and `geminiService.js` uses `@google/genai` to send the file and the analysis instruction to the Gemini API. The structured reply is written to `data/results.json` and later read back by the backend for the frontend to display. Authentication, a database and cloud storage are not part of the current architecture and are listed as future work.

3.2 SYSTEM REQUIREMENTS

3.2.1 SOFTWARE REQUIREMENTS

Table 3.1. Software requirements

Component	Role
Operating system	A Windows development environment is sufficient for the current local implementation.
Frontend runtime	Node.js environment running React and Vite.
Frontend technologies	React, Vite, JavaScript, HTML, CSS.
Backend runtime	Node.js with Express.js.
API style	REST API for frontend-backend communication.
File upload	Multer, for multipart audio and video uploads.
AI service	Google Gemini API.
AI SDK	@google/genai package.
Storage	Local JSON result file: data/results.json.
Development tools	Code editor or IDE, and a terminal or command prompt.
Network	Internet connectivity, required to reach the Gemini API.

3.2.2 HARDWARE REQUIREMENTS

Table 3.2. Hardware requirements

Component	Recommendation
Processor	Modern dual-core processor or better.
RAM	8 GB or more.
Storage	At least 5 GB free for project files, dependencies and uploaded test media.
Display	Standard desktop or laptop display suitable for web development.
Network	Stable internet connection for Gemini API requests.
GPU	Not required by the current architecture.

3.3 API DESIGN

The backend exposes a REST API between the React frontend and the Node.js application. The API accepts a meeting recording, processes it, stores the generated result and makes that result available to the frontend. The routes are declared in `meetingRoutes.js` and handled by the controller in `aiController.js`. The operations supported by the interface are summarised in Table 3.3.

Table 3.3. Input and output description of API operations

Operation	Input	Output / purpose
Meeting upload	Audio or video recording (multipart request)	Submits the recording for AI analysis
Meeting result retrieval	Meeting identifier	Returns the analysed meeting result
Meeting history retrieval	None / user context	Returns previously stored results

3.4 DEPLOYMENT ARCHITECTURE

The current implementation is intended for local development and demonstration. The React frontend runs on localhost:5173 and the Node.js and Express.js backend on localhost:5000. The frontend talks to the backend over HTTP, and the backend talks to the external Gemini API. Uploaded media is held temporarily by the backend and generated results are kept in the local file data/results.json.

This arrangement is not a production cloud deployment. Containerisation, a central database, authentication, monitoring, secure file storage and horizontal scaling are suitable future improvements. The present setup is sufficient to demonstrate the recording-to-analysis workflow while keeping the implementation manageable for this phase.

CHAPTER 4 PROJECT DESCRIPTION

4.1 METHODOLOGY

The project was built as a chain of connected modules that follow the life cycle of a recorded meeting. The frontend was created first to give users a meeting interface. The backend was then added to receive uploaded media and expose the REST API. Multer was introduced for file handling, after which the Gemini integration was connected through `@google/genai`. The AI response was shaped into a predictable JSON structure and linked to local result storage. Finally, the frontend was connected to the result flow so that the generated information could be displayed.

The methodology stresses separation of responsibilities. The frontend handles interaction and presentation. The backend handles requests, temporary files, communication with the AI service and result storage. Gemini performs the analysis, and the result file provides local persistence. This separation reduces coupling and gives clear places for later extensions.

4.2 MODULES

4.2.1 USER / MEETING MANAGEMENT MODULE

Purpose: To present meetings and their details in a form from which a recording can be submitted.

Input: User interaction with the React interface.

Processing: The interface offers Dashboard, Meetings, History and Profile views. A meeting page shows the meeting title, status, date, time, type (online or offline), number of participants, location, description, notes and the list of participants, together with a button for uploading the recording.

Output: A meeting-analysis request and, afterwards, the result view.

Technologies used: React, Vite, JavaScript, HTML, CSS.

Contribution: Gives the user a single place to find a meeting, see its context and start its analysis.

4.2.2 MEETING RECORDING UPLOAD MODULE

Purpose: To transfer a meeting audio or video file from the user to the backend.

Input: A recording selected by the user in the frontend.

Processing: The frontend sends the file in a multipart request. Multer in the Express backend receives it and writes it temporarily to the uploads directory. The configured

upload limit is 25 MB.

Output: A file stored on the backend and available to the next processing step.

Technologies used: React, Express.js, Multer.

Contribution: Provides the entry point of the whole pipeline and enforces the size limit.

4.2.3 AUDIO / VIDEO PROCESSING MODULE

Purpose: To make the stored recording available to the AI service.

Input: The uploaded media file in the uploads directory.

Processing: geminiService.js uses the @google/genai package to upload the file to Gemini. The platform relies on the multimodal model's own understanding of audio and video and does not run a separate local speech-recognition model in this phase.

Output: A reference to the uploaded media that can be supplied to the analysis request.

Technologies used: Node.js, @google/genai, Gemini API.

Contribution: Connects local file handling to the external AI service and keeps the speech-processing stage simple.

4.2.4 AI MEETING ANALYSIS MODULE

Purpose: To analyse the recording and return structured meeting information.

Input: The uploaded media and an analysis instruction that names the required fields.

Processing: The instruction asks Gemini for a summary, key points, decisions, action items and deadlines. The reply is requested in JSON form and is read by the backend before it is stored.

Output: A structured JSON object with the fields summary, keyPoints, decisions, actionItems and deadlines.

Technologies used: Gemini API, structured JSON, Node.js, Express.js.

Contribution: Converts an unstructured recording into data that the application can handle field by field.

4.2.5 MEETING SUMMARY MODULE

Purpose: To give the reader a short overview of the meeting and its key discussion points.

Input: The summary and keyPoints fields of the AI result.

Processing: The backend stores these fields, and the frontend displays the summary text and the list of discussion points in a dedicated section.

Output: A readable summary of the meeting.

Technologies used: Gemini API, Express.js, React.

Contribution: Lets a user grasp the meeting quickly without listening to the entire recording.

4.2.6 DECISION EXTRACTION MODULE

Purpose: To identify decisions or agreements reached in the meeting.

Input: The meeting recording processed by Gemini.

Processing: The analysis instruction requests decisions as a separate structured field, and the returned decisions are stored as part of the meeting result.

Output: A list of identified decisions.

Technologies used: Gemini API, structured JSON, Express.js, React.

Contribution: Keeping decisions apart from the general summary makes important outcomes easier to review.

4.2.7 ACTION ITEM EXTRACTION MODULE

Purpose: To identify the tasks that participants must perform after the meeting.

Input: The meeting recording and the AI analysis request.

Processing: Gemini identifies action-oriented statements and returns them as structured objects, each holding a task, an assignee and a dueDate.

Output: A structured list of action items.

Technologies used: Gemini API, JSON, Node.js, React.

Contribution: Turns spoken commitments into a form that can support later task-management features.

4.2.8 DEADLINE AND DUE-DATE EXTRACTION MODULE

Purpose: To preserve the time-related commitments made in the meeting.

Input: The meeting recording and the identified action items.

Processing: The AI output carries dueDate values inside the action items and a separate deadlines field for deadline information.

Output: Due dates attached to tasks and meeting-level deadline information.

Technologies used: Gemini API, JSON, Node.js, React.

Contribution: Explicit time information makes action items more useful for follow-up and prepares the ground for a reminder feature.

4.2.9 MEETING RESULTS MODULE

Purpose: To store and present the complete structured analysis.

Input: The JSON returned by Gemini.

Processing: The backend writes the result to data/results.json, and the frontend retrieves it and shows each field in its own section.

Output: A result view with the summary, key points, decisions, action items, assignees, due dates and deadlines.

Technologies used: Node.js, Express.js, JSON storage, React.

Contribution: Turns the AI response into a persistent and user-readable application result.

4.2.10 MEETING HISTORY AND TRACKING MODULE

Purpose: To give access to stored results and a basis for reviewing analysed meetings.

Input: Stored meeting-result records.

Processing: The backend and frontend use the result store to retrieve previously generated information, which the History view lists.

Output: Earlier meeting information, as far as the current implementation exposes it.

Technologies used: Node.js, Express.js, JSON storage, React.

Contribution: Maintains continuity between the first analysis and later review. Detailed task-progress tracking and reminders are not implemented in this phase.

Table 4.1 summarises the modules and their technologies.

Table 4.1. Module implementation summary

Module	Primary input	Primary output	Main technologies
User / Meeting Management	User interaction	Meeting analysis request and result view	React, Vite
Recording Upload	Audio / video file	Backend-accessible file	React, Express, Multer
Audio / Video Processing	Uploaded media	Gemini input	Node.js, @google/genai
AI Meeting Analysis	Meeting media	Structured JSON	Gemini API
Meeting Summary	AI result	Summary text and key points	Gemini, React
Decision Extraction	AI result	Decision list	Gemini, JSON
Action Item Extraction	AI result	Task, assignee and dueDate objects	Gemini, JSON
Deadline / Due Date	AI result	Due dates and deadlines	Gemini, JSON
Meeting Results	Stored JSON	Structured meeting display	Express, React
History / Tracking	Stored results	Previous meeting information	JSON, Express, React

CHAPTER 5 IMPLEMENTATION AND RESULT DISCUSSION

This chapter documents the implementation of the current prototype and discusses the results that can be established from it. Because no benchmark dataset, accuracy study, user study or latency experiment has been carried out, the discussion concentrates on implemented functionality, functional test design, observed system outputs and limitations rather than numerical performance.

5.1 FRONTEND IMPLEMENTATION

The frontend is a React application built with Vite, JavaScript, HTML and CSS, and it runs on localhost:5173. It provides the user-facing workflow and communicates with the backend through the REST API. The interface accepts the meeting recording, shows the progress of the analysis and presents the generated result in separate sections. The screens used in the demonstration (Section 5.7) are the meeting details and upload screen, the AI processing screen, the meeting summary screen and the screen listing decisions and action items.

5.2 BACKEND IMPLEMENTATION

The backend is built with Node.js and Express.js and runs on localhost:5000. It follows a modular structure made up of server.js, controllers, routes, services, an uploads directory and a data directory. The file aiController.js handles AI-related requests, meetingRoutes.js declares the meeting-related API routes, and geminiService.js integrates with Gemini. Uploaded files are held in the uploads directory, and generated results are stored in data/results.json.

The flow through the backend is straightforward: a route receives the request, the controller coordinates the operation, the service communicates with Gemini, and the result is written to the storage layer. This clear separation makes each part easy to maintain and extend.

5.3 APPLICATION ROUTES AND API OPERATIONS

The user moves through the application with the frontend routes listed in Table 5.1, which are served by the Vite application on localhost:5173. The numeric segment in a route identifies the meeting being opened. These routes call the backend REST API on localhost:5000, whose operations (upload, result retrieval and history retrieval) are described in Section 3.3 and declared in meetingRoutes.js.

Table 5.1. Application route summary (frontend, localhost:5173)

Frontend route	Description
/uploadrecording/{id} (e.g. /uploadrecording/2)	Upload the audio or video recording for the selected meeting
/airesults/{id} (e.g. /airesults/5)	Display the generated meeting result
/history	List previous meeting results
/Profile	Show profile details

5.4 GEMINI AI INTEGRATION

The AI layer uses the Google Gemini API through the `@google/genai` package. The backend uploads the meeting recording to Gemini and supplies an instruction asking for a structured meeting analysis. The design relies on the multimodal understanding of the model and does not add a separate local speech-recognition model in this phase. Google’s documentation describes audio understanding for tasks such as transcription, summarisation and question answering, and video understanding for extracting information from audiovisual content [11], [12].

The model configured for the project is Gemini 3.6 Flash. Model availability and API behaviour change over time, so the configured model identifier is kept in the project environment and should be recorded with any results that are reported. No claim about accuracy is made, since no project-specific benchmark has been run.

5.5 MEETING RECORDING UPLOAD

Uploading is implemented through the React frontend, the Express backend and the Multer file-handling layer. The user selects a supported audio or video recording and the frontend sends it to the backend. Multer receives the multipart request and stores the file temporarily, and the stored file then becomes the input to the Gemini upload step. The configured upload limit is 25 MB. Larger recordings are outside the tested scope of this phase.

5.6 AI PROCESSING WORKFLOW

The complete processing sequence is as follows:

- 1 The user uploads the meeting recording from the frontend.
- 2 The backend receives the multipart request and stores the file temporarily.
- 3 The backend invokes the Gemini service through `@google/genai`.
- 4 The recording is supplied to Gemini for analysis.
- 5 The analysis instruction requests the defined meeting-intelligence fields.

- 6 Gemini returns structured JSON.
- 7 The backend stores the result in data/results.json.
- 8 The frontend retrieves and displays the generated meeting information.

5.7 END-TO-END SYSTEM DEMONSTRATION

The demonstration follows one recording through the whole workflow, from selection in the upload interface to presentation of the structured result. Figures 5.1 to 5.4 are screenshots of the running application.

Figure 5.1 shows a meeting's detail page. It lists the title, status, date, time, type, number of participants, location, description, notes and participants, and it carries the Upload Meeting Recording button that starts the analysis.

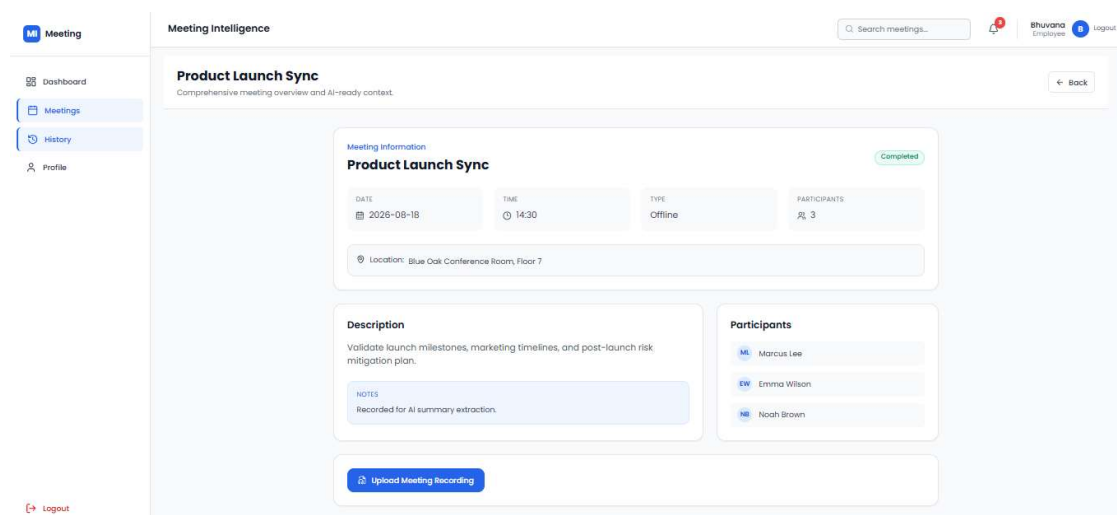


Figure 5.1. Meeting Upload Screen

Figure 5.2 shows the processing screen. A progress bar and a five-stage checklist (recording uploaded, uploading recording to AI, AI analysing meeting, generating meeting summary, AI processing completed) tell the user which step the analysis has reached.

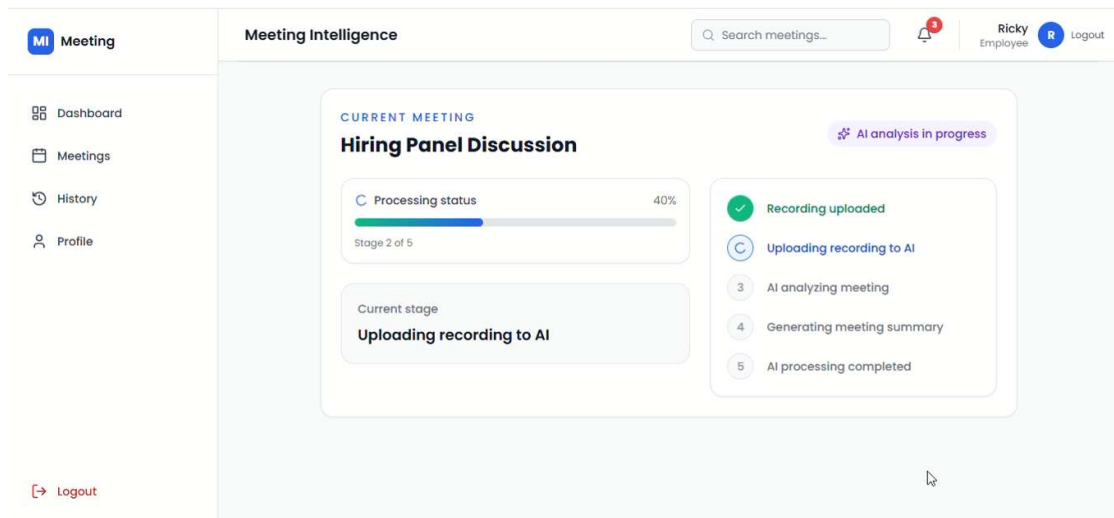


Figure 5.2. AI Processing Screen

Figure 5.3 shows the summary section of a processed meeting, with a status badge confirming that processing finished successfully.

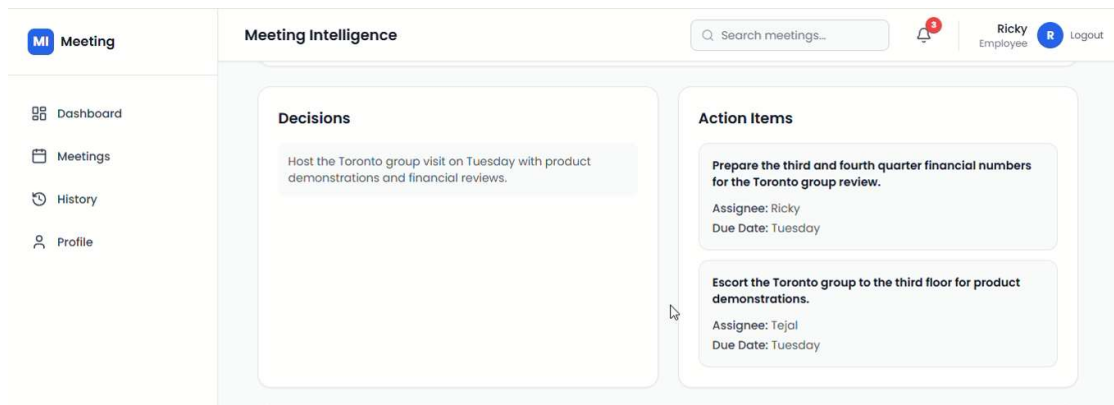


Figure 5.3. Meeting Summary

Figure 5.4 shows the decisions and action items extracted from a recording. Each action item lists the task, its assignee and its due date, in line with the structure defined in Section 5.9.

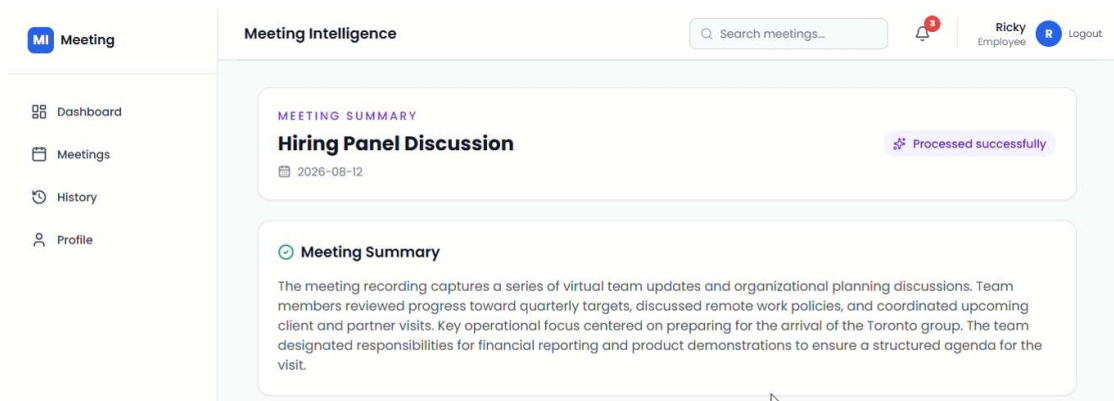


Figure 5.4. Action Items

5.8 FUNCTIONAL TESTING

Functional testing verified the major operations of the platform. It began by checking that the React frontend is reachable on localhost:5173 and that the Express backend is running on localhost:5000. Upload was then tested with valid audio and video files within the configured limit. After an upload, the system was checked to confirm that the recording reaches the backend and is processed by Gemini, and that the response contains the summary, key points, decisions and action items.

Action-item extraction was further tested with recordings that contain task-related discussion, to see whether the task, assignee and due date are identified when the meeting content states them. Result storage was checked by confirming that the generated result is written to data/results.json, and retrieval and display were checked by confirming that the stored information appears correctly in the frontend. Table 5.2 lists the test cases and the outcome expected from each. The behaviour observed for upload, processing, summary, decisions and action items is shown in Figures 5.1 to 5.4.

Table 5.2. Functional test cases

ID	Test case	Expected result
TC1	Open the frontend on localhost:5173	Application loads and the meeting interface is displayed
TC2	Start the backend on localhost:5000	Server runs and accepts API requests
TC3	Upload a valid recording within 25 MB	File is accepted and sent to the backend
TC4	Process the uploaded recording	Backend passes the file to Gemini and receives a response
TC5	Inspect the generated output	Summary, key points, decisions and action items are present
TC6	Use a recording with assigned tasks	Task, assignee and due date are identified where stated
TC7	Check result storage	Result is written to data/results.json
TC8	Retrieve and display the result	Stored information is shown correctly in the frontend

5.9 SYSTEM OUTPUTS

The output of the system is a structured meeting result and not a single block of text. Its primary fields are summary, keyPoints, decisions, actionItems and deadlines, and the actionItems structure holds task, assignee and dueDate. This organisation lets the frontend render each category as a separate section and gives later task-oriented features a foundation to build on. The fields are listed in Table 5.3.

Table 5.3. Output fields of the implemented AI result

Output field	Description	Status
summary	Overall summary of the meeting	Implemented
keyPoints	Important discussion points	Implemented
decisions	Decisions identified in the meeting	Implemented
actionItems	Action-oriented tasks extracted from the meeting	Implemented
actionItems.task	Task description	Implemented within actionItems
actionItems.assignee	Responsible employee or person	Implemented within actionItems
actionItems.dueDate	Due date associated with a task	Implemented within actionItems
deadlines	Deadline information identified in the meeting	Implemented

5.10 IMPLEMENTATION DISCUSSION AND OBSERVED ISSUES

The points below are implementation considerations and not measured failures. The project uses a 25 MB upload limit and temporary local storage. Because an external AI API is used, processing depends on network availability and on the behaviour of the configured model. The quality of the extracted information may also vary with recording quality, background noise, overlapping speech, ambiguous references and whether participants clearly state names and dates. These factors should be studied empirically and not assigned invented accuracy values. Table 5.4 summarises them.

Table 5.4. Observed issues and corrective considerations

Issue	Possible effect	Corrective consideration
25 MB upload limit	Large recordings may be rejected	Redesign upload handling and test larger media
Network dependence	AI processing needs access to Gemini	Add retry handling, clearer failure states and offline options
Recording noise	Speech may be harder to interpret	Evaluate preprocessing or dedicated ASR
Overlapping speech	Simultaneous speakers may reduce clarity	Evaluate speaker-aware processing and diarisation
Ambiguous names or dates	Assignee and due-date extraction may be uncertain	Add validation and user confirmation
Local JSON storage	Unsuitable for large multi-user persistence	Migrate to a database for production use
External LLM dependence	Output may vary across model or API versions	Pin and document the model configuration; validate the output structure

5.11 LIMITATIONS

- The system is recording-based and provides no real-time transcription.
- Direct integration with online meeting platforms is not implemented.
- Speaker identification is not implemented.
- Multilingual transcription is not a dedicated feature of the project.
- Email, calendar and document integrations are not implemented.
- Automatic task reminders are not implemented.
- Employee task-progress tracking is not implemented.
- Enterprise-grade authentication and role-based access control are outside the scope of this phase.
- Results are kept in local JSON storage and not in a production database.
- The upload limit is 25 MB.
- No accuracy percentage, user study, benchmark score or large-scale evaluation is reported, because none has been carried out yet.
- The system uses a hosted AI service, so processing depends on network availability and service configuration.

5.12 SUMMARY OF CONTRIBUTIONS

This phase demonstrates a working architecture for recording-based enterprise meeting intelligence. The frontend provides the meeting interface, the backend manages upload and API communication, Multer handles files, the Gemini service performs multimodal analysis, and the result is organised as structured JSON and stored locally. An end-to-end path from a meeting recording to structured meeting information is therefore in place.

The main contribution is the practical integration of these components in one workflow. The work does not claim a new speech-recognition algorithm, a new summarisation model or a statistically validated action-item extraction model. It is an application-level platform showing how a multimodal LLM can be connected to enterprise meeting-management needs and how the resulting information can be presented through a web application.

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The AI-Powered Enterprise Meeting Intelligence and Action Tracking Platform was designed to reduce the effort of turning recorded meetings into useful, structured information. The system accepts a meeting audio or video recording through a React and Vite frontend, passes it to a Node.js and Express.js backend, analyses the media with the Google Gemini API through @google/genai and stores the result in a local JSON file. The result is organised into a summary, key discussion points, decisions, action items, assignees, due dates and deadlines.

The architecture separates the presentation layer, the backend API, the AI-service integration, the file-handling layer and the result store. This makes the implementation easy to understand and a practical base for later enterprise capabilities. The project also addresses the real-world problem of employees who cannot keep complete meeting notes because of travel, noise, network conditions or the need to concentrate on the discussion.

The work should be read within its defined scope. It is not a real-time meeting assistant, does not join online meetings, does not identify speakers, does not send reminders and makes no measured accuracy claims. The value shown by this phase is the successful integration of recorded-media ingestion and AI-based structured analysis in a web application. More rigorous evaluation and enterprise-grade deployment remain important directions.

6.2 FUTURE WORK

The platform can be extended in several directions. The items below are planned work and are not features of the current implementation.

- **Real-time transcription** to process meeting content while the meeting is taking place.
- **Online meeting integration** so that recordings or transcripts are acquired automatically.
- **Offline support** through local or edge processing, reducing dependence on a network connection.
- **Multilingual transcription** for organisations that work in several languages.
- **Speaker identification** to link statements and action items to individual participants.

- **Email and document integration** to distribute results or combine them with related material.
- **Calendar integration** to connect due dates and follow-up activities with organisational calendars.
- **Task reminders** that notify responsible employees of approaching deadlines once configured.
- **Task-progress tracking** to extend action items into a full task life cycle.
- **Intelligent meeting search** across earlier meetings for decisions, topics, people and actions.
- **Enterprise authentication** for secure identity, access control and organisation-level management.
- **Cloud deployment** for central storage, scalable processing and access across devices.
- **Structured validation and confidence handling** so that uncertain AI outputs are highlighted for human confirmation.

Evaluation is a particularly important next step, and no accuracy values have been invented in this report. A future study should define a representative meeting dataset, prepare human-labelled reference summaries and action items, choose evaluation metrics and report results separately for summary quality, decision extraction, action-item extraction, assignee identification and due-date extraction. End-to-end latency should be measured across upload, AI processing, storage and frontend rendering, not only for a single internal stage.

REFERENCES

- [1] J. Carletta, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaiskos, W. Kraaij, M. Kronenthal, G. Lathoud, M. Lincoln, A. Lisowska, I. McCowan, W. Post, D. Reidsma, and P. Wellner, “The AMI Meeting Corpus: A Pre-announcement,” in *Machine Learning for Multimodal Interaction, Second International Workshop, MLMI 2005, Revised Selected Papers*, Lecture Notes in Computer Science, vol. 3869, Springer, 2006, pp. 28–39, doi: 10.1007/11677482_3.
- [2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” arXiv:2212.04356, 2022.
- [3] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” in *Advances in Neural Information Processing Systems* 33, 2020.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems* 30, 2017.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, vol. 1, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.
- [6] M. Zhong, D. Yin, T. Yu, A. Zaidi, M. Mutuma, R. Jha, A. H. Awadallah, A. Celikyilmaz, Y. Liu, X. Qiu, and D. Radev, “QMSum: A New Benchmark for Query-based Multi-domain Meeting Summarization,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.
- [7] Y.-N. Chen and D. Hakkani-Tür, “AIMU: Actionable Items for Meeting Understanding,” in *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, 2016, pp. 739–743.
- [8] M. T. R. Laskar, X.-Y. Fu, C. Chen, and S. B. TN, “Building Real-World Meeting Summarization Systems using Large Language Models: A Practical Perspective,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2023, pp. 343–352, doi:

10.18653/v1/2023.emnlp-industry.33.

- [9] F. Kirstein, T. Ruas, R. Kratel, and B. Gipp, “Tell me what I need to know: Exploring LLM-based (Personalized) Abstractive Multi-Source Meeting Summarization,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024, pp. 920–939, doi: 10.18653/v1/2024.emnlp-industry.69.
- [10] Gemini Team, “Gemini: A Family of Highly Capable Multimodal Models,” arXiv:2312.11805, 2023.
- [11] Google, “Gemini API: Audio Understanding,” Google AI for Developers, technical documentation, accessed September 2026.
- [12] Google, “Gemini API: Video Understanding,” Google AI for Developers, technical documentation, accessed September 2026.